Titanic Survival Prediction

Mathematics and the Training-to-Submission Pipeline

Kevin J. Medina October 24, 2026

1 The prediction problem

For each passenger, estimate the probability of survival from six recorded attributes. The target is binary: $y = 1$ means survived and $y = 0$ means did not survive. The labeled dataset contains 891 passengers; the separate Kaggle test dataset contains 418 passengers whose survival labels are hidden.

The model is logistic regression, trained with full-batch gradient descent. Its calculations are implemented manually with NumPy. The goal is a readable, reproducible predictive baseline whose mathematics can be followed from the raw observations to a submission.

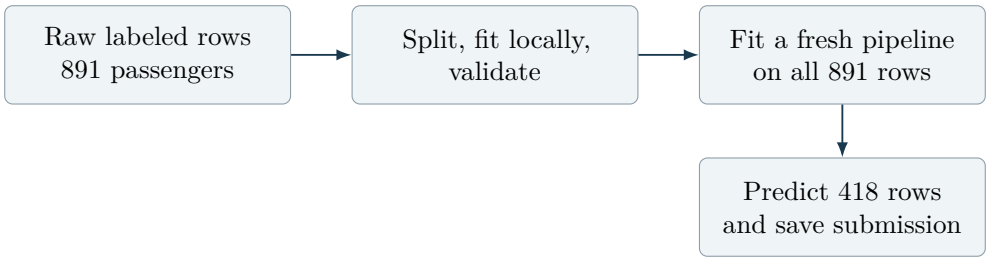
Represent every passenger in the same feature order

Position	Feature	Representation used by the model
1	Pclass	Original numerical class: 1, 2, or 3.
2	Sex	Fixed encoding: male = 0; female = 1.
3	Age	Median imputation, then standardization.
4	SibSp	Original count of siblings/spouses aboard.
5	Parch	Original count of parents/children aboard.
6	Fare	Median imputation, then standardization.

Only Age and Fare are standardized. Sex, Pclass, and the two count features remain unscaled. Treating Pclass numerically assumes an equal change in log odds between adjacent classes; this is a modeling choice.

With m passengers, each processed feature vector $x_i \in \mathbb{R}^6$ becomes a row of $X \in \mathbb{R}^{m\times 6}$. The labels form $y \in \{0,1\}^m$, the weights are $w \in \mathbb{R}^6$, and the bias is a scalar $b \in \mathbb{R}$. PassengerId is retained for matching predictions to passengers, rather than used as a feature.

The complete route



2 Learn preprocessing without leakage

Preprocessing learns information from data just as the model learns weights. To evaluate honestly, first split the *raw labeled rows*; only then fit the imputation and scaling statistics on the local training rows.

A reproducible, stratified validation split

Within each survival class, shuffle row positions using random seed 42. Reserve approximately 20% of each class for validation, rounding up within each class. The remaining passengers form the local training set:

$$\begin{aligned} 549 \text{ non-survivors} &\longrightarrow 439 \text{ training} + 110 \text{ validation}, \\ 342 \text{ survivors} &\longrightarrow 273 \text{ training} + 69 \text{ validation}. \end{aligned}$$

This produces 712 local training rows and 179 validation rows. Both classes appear in each set, and no passenger appears in both. The same seed and input order reproduce the same split.

Fit statistics on one designated training set

Let T denote the rows used to fit the current pipeline and $n = |T|$. For each continuous feature $j \in \{\text{Age}, \text{Fare}\}$, compute its median using *observed values in* T:

$$q_j = \text{median}\{a_{ij} : i \in T,\ a_{ij} \text{ is observed}\}.$$

Replace each missing value with this median:

$$\widetilde{a}_{ij} = \begin{cases} a_{ij}, & a_{ij} \text{ is observed}, \\ q_j, & a_{ij} \text{ is missing}. \end{cases}$$

Next compute the mean and *population* standard deviation from these imputed training values, including the filled entries:

$$\mu_j = \frac{1}{n}\sum_{i \in T} \widetilde{a}_{ij}, \qquad s_j = \sqrt{\frac{1}{n}\sum_{i \in T} (\widetilde{a}_{ij} - \mu_j)^2}.$$

Use $\bar{s}_j = s_j$ when $s_j > 0$ and $\bar{s}_j = 1$ when $s_j = 0$. The processed continuous feature is

$$\boxed{x_{ij} = \frac{\widetilde{a}_{ij} - \mu_j}{\bar{s}_j}.}$$

Standardization puts Age and Fare on comparable numerical scales, making a single gradient-descent step size more practical. The unit-scale fallback avoids division by zero for a constant training feature.

Transform unseen rows with frozen statistics

Validation rows use the local training values $(q_j, \mu_j, \bar{s}_j)$. Their own ages and fares never determine these statistics. After evaluation, the final pipeline learns new statistics from all 891 labeled rows; the 418 Kaggle test rows reuse those final values. The fixed Sex encoding and feature order remain identical throughout.

3 From a linear score to a stable loss

The score and the survival probability

For passenger i, form the linear score, or *logit*,

$$z_i = x_i^\mathsf{T} w + b, \qquad z = Xw + b\mathbf{1} \in \mathbb{R}^m.$$

The sigmoid turns this unrestricted score into a probability:

$$p_i = P(y_i = 1 \mid x_i; w, b) = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}.$$

Equivalently, $\log(p_i/(1-p_i)) = x_i^\mathsf{T} w + b$: the log odds are linear in the processed features. To avoid exponentiating a large positive number, evaluate the same sigmoid in two branches:

$$\sigma(z) = \begin{cases} \dfrac{1}{1+e^{-z}}, & z \geq 0, \\ \dfrac{e^z}{1+e^z}, & z < 0. \end{cases}$$

Both branches exponentiate a nonpositive number. Extreme scores can therefore produce probabilities near zero or one without exponential overflow.

Choose weights by Bernoulli likelihood

Under the model's conditional independence assumption, the likelihood of the observed binary labels is

$$\mathcal{L}(w, b) = \prod_{i=1}^{m} p_i^{y_i} (1 - p_i)^{1 - y_i}.$$

Maximizing this likelihood is equivalent to minimizing its negative mean log, the binary cross-entropy:

$$\begin{aligned} J(w, b) &= -\frac{1}{m} \sum_{i=1}^{m} \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right] \\ &= \frac{1}{m} \sum_{i=1}^{m} \left[\log(1 + e^{z_i}) - y_i z_i \right]. \end{aligned}$$

This is an unregularized objective: there is no additional penalty on weights.

Evaluate loss directly from the logits

For binary y_i, let $a_i = (1 - 2y_i) z_i$. Then

$$\boxed{J = \frac{1}{m} \sum_{i=1}^{m} \log(1 + e^{a_i}) \;=\; \frac{1}{m} \sum_{i=1}^{m} \left[\max(0, a_i) + \log(1 + e^{-|a_i|}) \right].}$$

The implementation uses a stable logarithm-of-exponentials operation to evaluate this expression. It avoids taking the logarithm of a rounded zero, avoids overflow in e^{a_i}, and avoids subtracting nearly equal large terms. No clipping of predicted probabilities is needed for the loss.

4 Derive and apply the training updates

Why the prediction error drives the gradient

Write the individual loss as $\ell_i = -y_i \log p_i - (1 - y_i) \log(1 - p_i)$. Using $\frac{dp_i}{dz_i} = p_i(1 - p_i)$ gives

$$\begin{aligned}\frac{\partial \ell_i}{\partial z_i} &= \left(-\frac{y_i}{p_i} + \frac{1 - y_i}{1 - p_i}\right) p_i(1 - p_i) \\ &= -y_i(1 - p_i) + (1 - y_i)p_i = p_i - y_i.\end{aligned}$$

Since $\partial z_i/\partial w = x_i$ and $\partial z_i/\partial b = 1$, the chain rule gives the mean-loss gradients

$$\boxed{\nabla_w J = \frac{1}{m} X^\mathsf{T}(p - y), \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(p_i - y_i).}$$

The vector $p - y \in \mathbb{R}^m$ contains one probability error per passenger. Multiplication by $X^\mathsf{T} \in \mathbb{R}^{6\times m}$ produces one gradient component per weight. Averaging by m matches the mean in the objective.

One epoch is one full-batch update

Start with $w^{(0)} = \mathbf{0}$ and $b^{(0)} = 0$, so every initial prediction is 0.5 and the initial loss is $\log 2 \approx 0.693147$. With learning rate $\eta = 0.1$, update all six weights and the bias together:

$$w^{(t+1)} = w^{(t)} - \eta \nabla_w J(w^{(t)}, b^{(t)}), \qquad b^{(t+1)} = b^{(t)} - \eta \frac{\partial J}{\partial b}(w^{(t)}, b^{(t)}).$$

Every update uses all rows in the current training set. Recompute the loss *after* the update so the reported value describes the current parameters. Log the initial loss, every 2,000 epochs, and the final epoch.

An explicit stopping rule

Define the improvement at epoch $t \geq 1$ by

$$\Delta_t = J(w^{(t-1)}, b^{(t-1)}) - J(w^{(t)}, b^{(t)}).$$

Stop when $0 \leq \Delta_t < 10^{-8}$ for 20 consecutive epochs, or after 20,000 epochs. Any other improvement resets that consecutive-epoch count. The test uses training loss only; validation does not choose when to stop.

A nonfinite loss or parameter triggers an error. An increase in loss greater than 10^{-12} also triggers an error, indicating that a smaller step size may be needed. Smaller numerical increases do not count toward convergence. Reaching the epoch limit is reported separately from meeting the stopping rule.

Small loss changes are a practical stopping signal, rather than a guarantee of an exact optimum or good performance on unseen passengers. The weights describe predictive associations under this model; they do not establish causal effects on survival.

5 Validate, refit, and create the submission

Evaluate once with frozen local parameters

Apply the local training statistics to the 179 validation passengers and hold the learned weights and bias fixed. For each transformed row, classify survival at the fixed probability threshold:

$$\widehat{y}_i = \mathbf{1}\{\sigma(x_i^\top w + b) \geq 0.5\} = \mathbf{1}\{x_i^\top w + b \geq 0\}.$$

Accuracy is the fraction of correct predictions:

$$\text{accuracy} = \frac{1}{179}\sum_{i=1}^{179} \mathbf{1}\{\widehat{y}_i = y_i\} = \frac{95 + 49}{179} = \frac{144}{179} \approx 80.45\%.$$

The confusion matrix uses actual labels as rows and predictions as columns:

	Predicted 0	Predicted 1
Actual 0	95 (true negatives)	15 (false positives)
Actual 1	20 (false negatives)	49 (true positives)

Observed results with the default settings

Model fit	Training rows	Epochs	Training loss
Local validation model	712	3,776	0.449862
Final submission model	891	3,850	0.442866

Both fits met the small-improvement stopping rule. The local model's *validation* loss was 0.424898; the table reports *training* losses. These measure different row sets and should not be interchanged.

Train a fresh final pipeline and predict the hidden labels

After recording local performance, start again from zero weights and bias. Fit the preprocessing statistics on all 891 labeled passengers, transform those rows, and train a new model with the same optimization settings. The earlier validation rows can now contribute because local evaluation is complete.

Freeze this final pipeline. Transform each of the 418 Kaggle test passengers using its training-derived medians, means, and scales, then apply the same 0.5 threshold. Write one row per test passenger, preserving test row order, to *submission.csv*. Its two columns are *PassengerId* and *Survived*; predictions are integer 0 or 1, with no extra index column.

What the result establishes

The validation result is a development estimate for this particular split. Repeated tuning against it can make that estimate optimistic. Numeric Pclass, linear log odds, the selected six features, and median imputation are simplifying assumptions. The Kaggle test labels are unavailable locally, so the submission has no locally measured test accuracy or claimed Kaggle score.